EEET2601 Engineering Computing 1

Functions

Quang Tran

RMIT UNIVERSITY

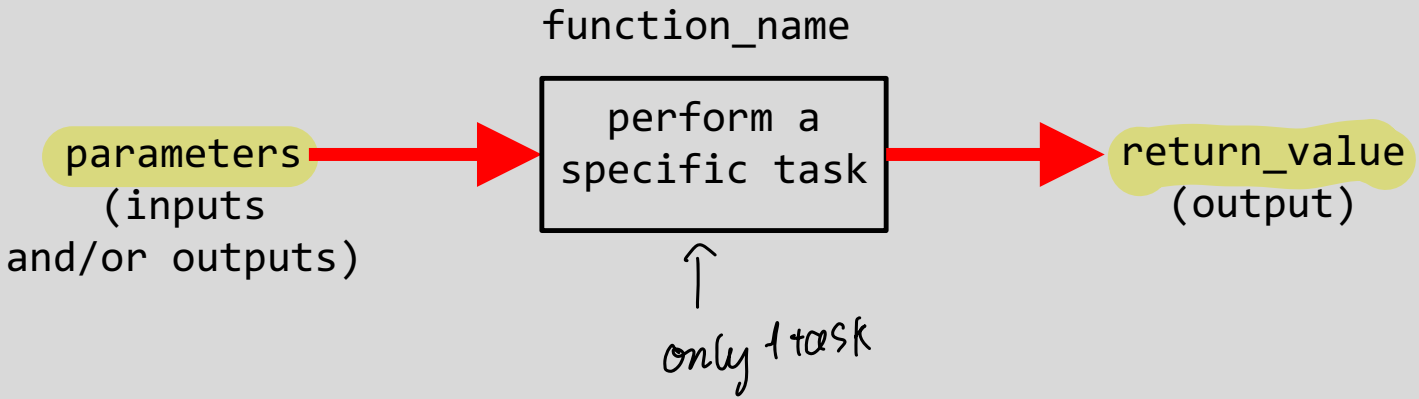
What is a function?

- A function is a block of code which performs **a specific task**
- When writing programs, we have used some **built-in functions** from C standard libraries such as printf(), scanf(), getchar(), strlen(), strcmp(), etc.
- Built-in functions are written by other people
- To use them, we must include the library they belong to at the beginning of the file, e.g. `#include <stdio.h>`

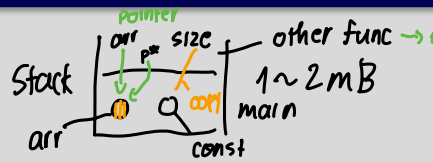
Built-in vs user-defined functions

- Built-in functions are written by other people
- To use them, we must include the library they belong to at the beginning of the file, e.g. `#include <stdio.h>`
- What if none of the built-in functions provide with what you need?
- We can write our own functions => **user-defined** functions

Analyze a function



Define a function



stack → last in Fist out
LIFO

```
<return_type> function_name(parameters) {
    // code to perform a specific task
    return return_value;
}
```

function header

function body

return statement

2 type of passing
perimeter → copy value
reference → pointer

call

print (argument , argument) ;

Define a function

- A function works like a mini program which accepts some inputs to perform a specific task and returns some outputs
- Both inputs and outputs are optional
- The data type of return_value must match return_type
- If a function has no outputs, return_type is **void** and the return statement is omitted
- The parameters in the function header are called **formal parameters** (or **parameters**)

Use a function

- To use a function, we call it with the formal parameters replaced by **actual parameters** (or **arguments**)
  - ✓ e.g. float sq = square(5); // 5 is the actual parameter
- The value of the actual parameter is copied to the formal parameter
- Formal parameters are local variables of the function. They are created upon entry into the function and destroyed upon exit

Example: Define then use a function

```c
#include <stdio.h>

// Function definition
float square(float num) {
    num = num * num;     // Calculate square of num and store it in num
    return num;          // Return the result
}

int main(int argc, char const *argv[]) {
    float sq = square(5);    // Call square() and store the result in sq
    printf("%f\n", sq);
    printf("%f\n", square(7.2));   // Call square() as a parameter of printf()

    return 0;
}
```

Example: Declare, use, then define a function

```c
#include <stdio.h>

// Function declaration
float square(float num);    // Only function header, no body

int main(int argc, char const *argv[]) {
    float sq = square(5);    // Call square() and store the result in sq
    printf("%f\n", sq);
    printf("%f\n", square(7.2));   // Call square() as a parameter of printf()

    return 0;
}

// Function definition
float square(float num) {
    num = num * num;    // Calculate square of num and store it in num
    return num;         // Return the result
}
```

General program structure

```
// include libraries
…

// function declarations for main
…

int main() {
    …
}

// function definitions for main
…
```

Why function?

1. A large C program can be constructed by having several functions calling each other => functions are the **building blocks** (like lego blocks) of procedural programming (also known as structural programming or imperative programming)
2. Functions make the program more readable and easier to understand => **easy to edit and debug**
3. Once defined, a function can be called many times without writing the same code again => **reusability**

What is a well-designed function?

1. Only does one thing and does it well
2. Almost always returns a value
   - Think at least 3 times if you design a function which doesn't return anything (void function)
3. Can be viewed entirely in one screen without scrolling
   - This often means no more than 30 LOC (Lines Of Code)

Local vs global variables

- **Local variables**: variables declared inside a function, or in the list of input parameters of a function, or in a block of code surrounded by a pair of {}
  - They are only understood by that function or that block of code
- **Global variables**: variables declared outside all functions
  - They are understood by all functions

Local vs global variables

- Local variables are not initialized by the system. You must initialize them by yourself
- Global variables are initialized automatically by the system
  - int => 0
  - char => '\0'
  - float => 0
  - double => 0
  - pointer => NULL

Scope rules

1. Local variables are created upon entry into the function or a block of {} and destroyed upon exit
2. Global variables hold their values throughout the lifetime of a program and can be accessed by any functions defined in the program
3. You can use a same name for local and global variable. However, the value of local variable inside a function of a block {} will take preference

Example: Scope of local variables

```c
#include <stdio.h>

int main(int argc, char const *argv[]) {
    // Declare and initialize local variables
    char c = 72;
    int i = 218;
    float f = 7.23;

    // Local variables can only be used inside the function they are declared
    printf("c = %c, i = %i, f = %f\n", c, i, f);

    return 0;
}
```

Example: Scope of global variables

```c
#include <stdio.h>

float global_var;

int main(int argc, char const *argv[]) {
    // Declare and initialize local variables
    char c = 72;
    int i = 218;
    float f = 7.23;

    // Update g
    global_var = c + i - f;

    // Global variables can be used in any functions
    printf("c = %i, i = %i, f = %f, global_var = %f\n", c, i, f, global_var);

    return 0;
}
```

Example: Variables with a same name

```c
#include <stdio.h>

int num = 5;

int main(int argc, char const *argv[]) {
    // Declare and initialize local variables
    int num = 11;
    float f = 7.23;
    float sum = num + f;

    // Global variables can be used in any functions
    printf("num = %i, f = %f, sum = %f\n", num, f, sum);

    return 0;
}
```

References

1. Fresh2Refresh, C Programming Tutorial, 2019.
2. StudyTonight, C Programming Language Tutorial, 2019.